

Image-based Clone Code Detection and Visualization

Yafang Wang^a, and Dongsheng Liu^b

College of Computer Science and Technology, Inner Mongolia Normal University, Hohhot 010022, China.

^a15540931820@163.com

^bCorresponding author: lds@imnu.edu.cn

Abstract—Currently, research mainly focuses on four perspectives of text, vocabulary, grammar and semantics in the field of clone code detection. However, few breakthroughs have been made in the effect of clone code detection for a long time. For this purpose, a new image-based clone code detection and visualization (ICCV) is proposed with the inspiration of image processing. First, the source code is preprocessed by removing comments, whitespace, etc. from which a "clean" function fragment can be obtained, and the identifiers, keywords, etc. in the function can also be highlighted; Then the processed source code is converted into images and these images are normalized; Finally, Jaccard distance and perceptual hash algorithm are used to detect and visualize the clone code information. In order to verify the validity of the experiment, six open source software were used to constitute the evaluation data set for testing. The experimental results show that ICCV can detect 100% type-1 clone code, 88% type-2 clone code and 60% type-3 clone code, which proves the good effect of ICCV on clone code detection.

Keywords—clone code; clone detection; visualization; Jaccard distance; perceptual hash.

I. INTRODUCTION

Copying code snippets, and then reusing the code by pasting and adjusting (such as adding, deleting, or modifying statements) is a common practice in software development. This reuse mechanism causes a large number of identical or similar code fragments in the code repository, namely clone code. In addition, using a particular development framework or reusing design patterns can also result in clone code. Clone code is an important research content in the field of software engineering, which is embodied in many fields such as software maintenance, software evolution, software quality, software reuse and software authorization, and anti-plagiarism. Empirical studies have shown that a software system may have 20% to 30% of clone code[1][2][3][4], sometimes even up to 50%[5]. A large number of clone code will have a negative impact on the software system[6], for example, repeated use of code snippets containing unknown errors may lead to error propagation, reduce the reliability of the software system, and so on. Therefore, the detection, identification and presentation of clone code in the software system can help developers to identify and better understand the clone code, thus reducing the extra labor cost and the maintenance cost of the software. In order to solve this problem, many good code clone detection methods have been proposed in academic. However, these methods need to convert the source code into Token, AST (Abstract Syntax Tree) or PDG(Program Dependence Graph) according to their own

rules, such as strict rule definition, complex intermediate processes, and there is no uniform standard for source code standardization. Currently, the commonly used visualization technologies of clone code detection data include scatter plot, coverage tree map, bar chart, etc., but these methods are only applicable to the clone data presentation of small scale systems, which cannot reflect the complexity of clone code in software systems with large data volume, and cannot give play to the due value of detected data. For this purpose, this paper proposes a Code Clone detection based on Image Similarity and Visualization (ICCV). In the development process, the first impression programmers face when looking at source code in IDE during development are visual images of code. To manually look for clones in their software projects, one roughly scans the code by looking for similar shape and layout of two source code fragments. Once a similar piece of code is found visually, one can perform a more fine-grained checking of the source code text, its syntax, or its behaviours. We follow that intuition of using visual similarity between source code images to look for real clones. This paper follows the intuition of visual similarity between source code images to find clones. Clone codes are determined by representing source code fragments as images, detecting similarity between images, and using Force Atlas2 algorithm to visualize code clone data. The code clone detection method proposed in this paper converts the code into images for detection. Compared with other traditional detection techniques with strict definition of transformation rules, this method is more flexible and robust, and provides a new perspective for code cloning detection research on the representation of source code.

II. RELATED WORK

Scholars have been studying clone code since more than 20 years ago[7]. In this field, the classification standard widely recognized by researchers at present is the four-category standard proposed by Bellon et al. according to the degree of clone code[8]. Type-1 refers to a code pair that removes spaces and comments exactly the same; Type-2 refers to code pairs that are identical except for type names, identifiers, and constants; Type-3 refers to a pair of code where some statements are added, deleted, modified, or where identifiers and types are replaced, but the syntactic structure is basically the same; Type-4 refers to code pairs that are semantically similar but have different syntactic structures.

A. Clone Code Detection method

At present, there are many excellent code cloning

detection methods in the academic circle[4][9][10][13][16][17] [19][20], which are mainly Text-based, Lexical-based, Grammar-based and Semantics-based detection method.

Text-based clone code detection methods mainly include NiCad[9], SDD[10], Dup[11], Duploc[12] and so on. NiCad first preprocesses the source code, then converts it according to specific rules, and finally uses the dynamic matching mode to find the longest common subsequence, and finally can detect the clone code of type-1 to type-3. SDD directly establishes an inverted index on the source code, and uses the n-Nearest Neighbor algorithm for detection, which can detect clone code from type-1 to type-3. However, since the source code itself has meaning, you will lose a lot of information by treating it as just text.

Lexical-based clone code detection technologies mainly contain CCFinder[4], CP-Miner[13], CCLearner[14], Boreas[15], etc. CCFinder converts the source code into regularized sequences and parameterized symbols according to certain rules, and then uses the suffix tree matching algorithm for detection. CP-Miner uses frequent sub-item mining technology to detect large-scale systems, both of which can detect type-1 and type-2 clone code. The lexical approach to representing code ignores the syntax and semantic information of the source code.

Gramm-based clone code detection methods mainly include Deckard[16], CloneDR[17], CDLH[18] and so on. The principle of Deckard and CloneDR is to parse the source code into AST, and then use the tree matching technique and locally sensitive hash matching algorithm to compare the similarity. Deckard can detect type-1 to type-3 clone code, and CloneDR can detect clone fragments of type-1 and type-2. Due to the complexity of tree matching algorithm, the computational overhead of this method is relatively large.

Semantic-based clone code detection technologies mainly include Duplix[19], ConQAT[20], and the method of clone detection by using slice of isomorphic program dependent graph[21]. Duplix generates program dependency graph for source code, and uses the isomorphic program dependency graph subgraph matching method of post-order section for detection, and clones from type-1 to type-4 can be obtained. ConQAT symbolizes the source code and USES the detection algorithm based on the suffix tree and index to detect type-1 and type-2 clone code. The algorithm used by graph-based detection technology has high space-time complexity and computational overhead.

B. Clone data visualization

Due to the large amount of detected clone code data, it is not conducive to developers to understand the complex clone information in the source code, therefore, it is necessary to visualize the clone code and present it to developers in a graphical manner. Most of the existing visualization methods appear as accessory tools of clone code detection technology, including Coverage Tree Map, Scatter Plot, Line Chart, Bar Chart and so on.

Coverage tree map can be used to show the distribution of clone code in the source code. Clone Detective¹ in Visual Studio is using this technique to visualize clone

information. In the coverage tree map, the file is expressed as a rectangular block, through the size of the rectangular block, the positional relationship between the rectangular blocks, and the color of the rectangular block to indicate the number of lines of code in the file, the inclusion relationship between codes and the coverage of code cloning in the file, move the cursor to the rectangular block, also can display more detailed code information. However, coverage tree map does not reflect the relationship between clone code.

Scatter plot can show the relationship between clone code. The CCFinder proposed by Kamiya et al adopts scatter plot. In a scatter plot, the horizontal and vertical coordinates are all code elements, and the clone relationship between code fragments is represented by points or lines. But when the scale of the system is large, the effect of the graph is very compact, and no other information can be obtained from it except which module has a large clone proportion.

Line chart can only reflect the information of a clone fragment. The horizontal and vertical coordinates are respectively the number of lines where the code is cloned and the location of the file where the segment is. Bar chart can show the distribution of clone group in the file, with each column representing a file and each clone group represented by a color. Clone Visualizer[22] proposed by Zhang et al. used line chart and bar chart to present clone code. However, these two methods only apply to display of less clone data.

III. IMAGE-BASED CLONE CODE DETECTION AND VISUALIZATION

This paper proposes a code clone detection based on image similarity and visualization method (ICCV). Its flow chart is shown in figure 1, which consists of five core steps: (1) Remove the non-function code such as comments and whitespace in the source code, and extract the code function fragments; (2) Add highlight to the preprocessed code snippet and convert it to image; (3) Crop, fill and resize image; (4) The standardized code image is detected using the Jaccard distance and perceptual hash algorithm to obtain clone code information and return the result; (5) Visualization of clone code data based on FA2.

A. Pre-processing

Source code preprocessing is the basic work of image-based clone code detection and visualization. Based on the Python language as the research object, the source code preprocessing steps mainly include removing the non-function code such as single-line comment, multi-line comment and white space in the source code, identifying and marking the code fragment according to the function granularity, highlight the code based on the key weights of the keyword, data type, function name, identifier, and number or string in the code.

Image-based clone code detection and visualization is to calculate the similarity according to the pixel distribution matching in the source code image. If two code fragments are cloned, most of the code pixels are aligned between them, and vice versa. Therefore, code preprocessing will directly affect the subsequent process and detection results.

¹ <https://github.com/terrajobst/clonedetective-vs>

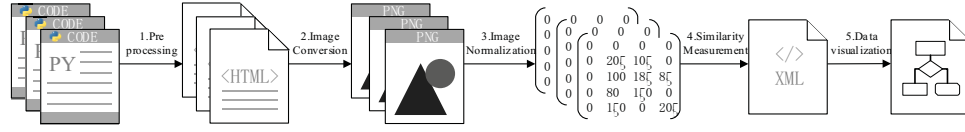


Figure 1. Overall research route.

The functions extracted in this paper include the loop nesting depth, not just the information on the lexical level [23]. The main steps of the function extraction algorithm are as follows:

Algorithm 1 Find Function Algorithm.

```

function FINDFUNCTION(pyFile)
    F = [], subF = [], Fflag = False, subFflag = False
    for each line ∈ all lines do
        if "def" in each line and not Fflag then
            F.append(each line[pos:])
        end if
        if Fflag then
            if "def" in each line and not subFflag
then
                subF.append(each line[pos:])
            end if
            if subFflag then
                subF.append(each line)
            end if
        else
            F.append(each line)
        end if
    end for
    return F, subF
end function

```

B. Image Conversion

The preprocessed source code needs to be converted into images, and then according to the calculated similarity between images, clone code detection is carried out to determine the code clone pair. In this paper, with the help of **text processing tool Highlight**, function codes extracted by preprocessing are added in batches to be highlighted and converted into HTML files, so that the codes with high weight play a role in image similarity detection, thereby reducing the proportion of unimportant codes such as numbers and strings in the results.

This paper uses Python imgkit to convert the HTML files for each function in the file to Portable Network Graphics (PNG). The image is read into storage in a two-dimensional matrix of size $m \times n$. Each value in the matrix ranges from 0 to 255, representing the 8-bit grayscale image of the original image. The grayscale value is obtained by averaging the red, green and blue (RGB) values in the color channel of pixel points.

C. Image Normalization

The graphical function code fragments cannot be directly used for similarity detection, which is due to the difference in pixels and proportions between directly converted images, which does not meet the detection conditions of Jaccard distance and perceptual hash algorithm. In order to solve this problem, the source code converted image needs to be normalized.

In this paper, the image background, that is, the non-

source pixel is set to black, and its gray value is 0, while the pixel of source code is calculated by averaging the RGB value of the pixel point, which means that the pixel number of source code contained in the image can be calculated according to the number of non-zero elements in the matrix.

When the image similarity detection is performed, the original images to be detected have the same size with each other and cannot be scaled at will, so as to avoid the dislocation of pixel points in the image and the loss of clone pairs due to the different font sizes of the code in the image. Based on this, all images need to be normalized. For images of different lengths, use black to complement a relatively short image to be equal to the length of the longer image. Since the generated image width is uniform pixels, in order to reduce the problem of excessive similarity due to excessive proportion of non-code pixels in the pHash detection process, it is necessary to greatly cut non-code pixels in width, and also ensure that the widths between the images are the same. The main steps of the image normalization algorithm are as follows:

Algorithm 2 Normalize Image Size Algorithm.

```

function UNIFORMIMAGESIZE(pngList)
    W = []
    for each png ∈ pngList do
        if maxH < png.height then
            maxH ← png.height
        end if
        img ← png.convert(grayscale)
        img ← img.array
        for i in range(img.height) do
            for j in range(decrease(img.width)) do
                if img[i][j] ≠ 0
                    W.append(j + 1)
                end if
            end for
        end for
        width ← max(W)
        if maxW < width then
            maxW ← width
        end if
    end for
    return maxH, maxW
end function

```

D. Similarity Measurement

There are two algorithms of calculating image similarity: **Jaccard Distance** and **perceptual hash (pHash)**.

Using Jaccard distance detection code to clone fragments is relatively fast and can process a large amount of data, but the effect is not ideal. This method can only detect simpler clone code, which cannot be accurately identified for more complicated situations. Therefore, pHash algorithm is used for further detection. the pHash algorithm preserves the low-frequency part of the image to the greatest extent by **discrete cosine transform**. As long as

the overall structure of the image remains unchanged, its corresponding hash value is basically unchanged, which can better identify relatively complex clones situations.

1) Jaccard Distance

Jaccard distance[24][25][26][27] is an indicator used to measure the difference between two sets. It is a complement of Jaccard similarity coefficient, defined as 1 minus Jaccard similarity coefficient. In order to calculate Jaccard distance, the difference between the two code images based on zero-norm is given as follows. Two equal-sized images are given, respectively represented by two-dimensional matrices A and B of size $m \times n$. Matrix D is a element-wise difference matrix derived from matrices A and B, where the elements

```
def bubble_sort1(arr):
    n = len(arr) - 1
    for i in range(n, 0, -1):
        count = 0
        for j in range(0, i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                count += 1
        if count == 0:
            return
    return arr
```

Fig.2(a)bubble_sort1

```
def bubble_sort2(arr):
    pos, k = 0, len(arr) - 1
    for i in range(len(arr)-1):
        flag = 0
        for j in range(0, i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                flag, pos = 1, j
        k = pos
        if flag == 0: break
    return arr
```

Fig.2(b)bubble_sort2

```
def bubble_sort(arr):
    n = len(arr) - 1
    for i in range(n, 0, -1):
        count = 0
        for j in range(0, i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                count += 1
        if count == 0:
            return arr
```

Fig.2(c)diff

```
def bubble_sort(arr):
    n = len(arr) - 1
    for i in range(n, 0, -1):
        count = 0
        for j in range(0, i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                count += 1
        if count == 0:
            return arr
```

Fig.2(d)sum

Figure 2. Code segment

In this paper, the matrix obtained by adding matrix A and B is denoted as matrix S. S_{ij} represents the elements of the i -th row and the j -th column in matrix S. The area of source code text contained in these two images is represented by the number of non-zero elements in the matrix S, and is denoted as sum. In order to limit the distance between (0,1), this paper uses sum to normalize the distance. As shown in figure 2(d), the value of sum is the number of non-zero pixels in the yellow-boxed areas of the figure.

$$S = A + B \quad (3)$$

$$sum = \sum s_{ij} (s_{ij} \in S | s_{ij} \neq 0) \quad (4)$$

Since the background occupies a large proportion in the image, and the backgrounds always match each other, in order to prevent the influence caused by this situation, the distance value far smaller than the actual situation is generated, this paper selects the sum of the number of pixels in the code area, not the total number of pixels in the image. Then the normalized distance is calculated according to the diff value and sum value, and the similarity is complementary to the distance. The normalized distance between matrices A and B is denoted as distance, and the similarity between the two images represented by matrices A and B is denoted as similarity.

$$distance = diff / sum \quad (5)$$

$$similarity = 1 - distance \quad (6)$$

2) Perceptual hash

In this paper, the perceptual hash algorithm [28][29] is used for clone detection of code image. PHash algorithm uses Discrete Cosine Transform (DCT) to convert the image in the pixel domain into the frequency domain, and obtain

of the i -th row and the j -th column are denoted as d_{ij} . The difference between the two images is represented by the number of non-zero elements in matrix D, called diff. As shown in figure 2(a) and figure 2(b), given a pair of source code images bubble_sort1 and bubble_sort2, their differences is shown in figure 2(c), and the diff value is the number of non-zero pixels in the red-boxed areas of the image.

$$D = A - B \quad (1)$$

$$diff = \sum d_{ij} (d_{ij} \in D | d_{ij} \neq 0) \quad (2)$$

the matrix of the frequency coefficient. Select the upper left area elements to calculate the image hash value. By comparing the distance between the hash values of two images, we can judge whether the images are similar or not. The main steps of pHash algorithm are as follows:

Algorithm 3 pHash Algorithm.

```
function pHash(picture)
    H = [], picture.size. reduce( $n \times n$ ), picture ←
    picture.convert( grayscale )
    Matrix_ ( $n \times n$ ) ← DCT( G_picture ), matrix_ ( $k \times k$ )
    ← Top_left( Matrix_ ( $n \times n$ ) )
    avg ← average( Matrix_ ( $n \times n$ ) )
    for i in range(n) do
        for j in range(n) do
            if matrix[i][j] ≥ avg then
                H.append(1)
            else
                H.append(0)
            end if
        end for
    end for
    return H
end function
```

The maximum pHash distance is used to normalize the calculated distance, and the similarity and distance complement each other. The pHash distance between matrix A and B is expressed by $pHash(A, B)$. The maximum pHash distance between any two matrices is denoted as $pHash_{max}$, and the similarity between the two images represented by matrix A and B is expressed by variable similarity.

$$similarity = 1 - pHash(A, B) / pHash_{max} \quad (7)$$

E. Data visualization

This paper selects the ForceAtlas2 (FA2) [30] layout

algorithm to visualize the detected clone code information. ForceAtlas2 is a force-directed algorithm that combines the characteristics of several well-known network layout algorithms [31][32][33], which spatializes the network by simulating physical systems. When the algorithm is running, the nodes repel each other like charged particles, while the edges attract the nodes like springs. When the algorithm is stable, it will reach the physical equilibrium state, resulting in a stable layout. In this paper, clone code fragment is set as node, clone relationship is edge to represent the relationship between clone codes, and the network analysis software Gephi is selected as the drawing tool to extract the clone fragment name and clone relationship for network drawing. The main steps of extraction algorithm are as follows:

Algorithm 4 Extract Clone Information Algorithm.

```

function EXTRACTION(result_xml)
    node = {}, edge = {}, N = [], file = [], clone_file =
    [], flag = FALSE
    for each line ∈ all lines do
        if "similarity" in line then
            N.append(clone_similarity)
        end if
        if "file" in line then
            if node.get(file_name) is None
                node[file_name] ← 0
            end if
            node[file_name] ← node[file_name] + 1
            if Fflag then
                file.append(file_name)
                flag ← TRUE
            end if
            elif flag then
                clone_file.append(file_name)
                flag ← FALSE
            end if
        end if
    end for
    for i in range(N.length) do
        edge[file[i]] ← {clone_file[i] : N[i]}
    end for
    return node, edge
end function

```

IV. EXPERIMENTAL ANALYSIS

A. Data set

Since the methods proposed by the researchers are not uniform in terms of detection granularity, language orientation and system size, there is no international standard test data set for evaluating clone code detection tools. Svajlenko et al. proposed a JAVA code set, BigCloneBench[34], but it is limited by language and only ten functions, and cannot be applied to all detection tools. In order to accurately and objectively evaluate ICCV, this paper draws on the variation insertion[35] test method proposed by the Roy team. This method does not rely on any tools or programming language, can be more fair and independent evaluation of test results. The core idea of it is artificially in a given source code snippet to modify, add or delete some statements, generate code cloning for type 1 to type 3. Such as the source code to change the type name,

modify the constant value, add whitespace, and other operations, generate a type-2 code pair. All source code fragments obtained after mutation insertion are mutation code fragments. In this process, all mutation related information is recorded to facilitate the calculation of recall and precision of detection results of the proposed method. The data set construction and mutation related information recording personnel include the author, 5 intra-school clone code research experts and 6 enterprise software developers. This paper takes function granularity as the research object and selects 219 source code fragments from 6 open source projects.

The specific mutation insertion process includes three cases. The first case is to add, delete white space character or comment on the source code fragment, and modify the annotation operation, so that the inserted code fragment and the source code fragment form a type-1 clone pair. The second case is to modify the source code fragment data type, identifier and constant value and other operations, so that the source code fragment and mutation inserted code fragment form a type-2 clone pair. The third case is to delete, modify or add a small number of statements in the source code fragment, and the modified code fragment has the same function as the source code fragment, so that form a type-3 clone pair. After mutation insertion, 275 mutation fragments were obtained, so there were 494 clone code fragments in the data set. In order to make the results more objective, on this basis, add 72 non-clone code fragments, namely noise fragments, and any two noise fragments are not clone. The basic project information is shown in table I.

TABLE I. PROJECT BASIC INFORMATION

Project	Version	Project size/row	Basic function
pandas	0.24.0	384384	Data structures and data analysis tools
scipy	1.2.0	318131	Scientific computing tool
django	2.1.4	333179	A rapidly developed high-level Python Web framework
pytorch	1.0.0	244439	End-to-end deep learning platform
scikit-learn	0.20.2	230534	Data mining and data analysis tools
keras	2.2.4	66156	Advanced neural network API

B. Evaluation index

The detection result is a dichotomy problem, that is, the code fragment is a clone fragment or not a clone fragment. Therefore, the recall rate and precision are adopted to evaluate the experimental results [23][36][37]. Recall refers to the ratio of all clone code detected to the total number of clones.

$$Recall = \frac{TP}{TP + FP} \quad (8)$$

Precision refers to the proportion of real clone code detected by the clone detection algorithm among candidate code clones.

$$Precision = \frac{TP}{TP + FN} \quad (9)$$

TP represents the intersection of the clone fragment detected by ICCV and the real clone code fragment. FP

stands for the collection of clone code fragments detected by ICCV that are clone fragments but are not real clone fragments. FN represents the collection of real clone code fragments not detected by ICCV.

```
<Clones>
<clone nlines='9'>
<source file='django/django/contrib/sessions/backends/db.py' startline='49' endline='57'></source>
<source file='django/django/contrib/sessions/backends/file.py' startline='101' endline='109'></source>
</clone>
<clone nlines='11'>
<source file='django/tests/csrf_tests/tests.py' startline='650' endline='660'></source>
<source file='django/tests/csrf_tests/tests.py' startline='761' endline='771'></source>
</clone>
<clone nlines='8'>
<source file='django/tests/syntax_tests/i18n/test_filters.py' startline='40' endline='47'></source>
<source file='django/tests/syntax_tests/i18n/test_get_language_info_list.py' startline='39' endline='46'></source>
</clone>
</Clones>
```

Figure 3. Clone code detection result

In the experiment, 494 clone code fragments and 72 non-code clone fragments were integrated together as a project to be tested, and then using ICCV for detection. According to the detection results, there are 414 clone code fragments in the data set. In order to verify whether the detected clone code fragments are evenly distributed among different projects, table 2 shows the distribution of 494 real clone code fragments in the source project and 414 detected clone code fragments in 6 projects.

According to the analysis of the error detection and missed detection in table II, it is found that the main reason for the missed detection is because of these fragments and its corresponding clone code fragment in the replacement of identifier length which can lead to great changes have taken place such as a large number of pixels to the overall migration occurred dislocation, and includes some statements add delete, resulting in the overall structure of the code snippet contour change, when detecting similarity between code images, only a few pixels can match. The main reason for the error detection is that the two code fragments are not clone pair, but their code structure is similar in outline, so that most of the pixels can match each other, caused by calculating distance between two images is smaller than the actual distance, this kind of situation is relatively rare, the algorithm will be improved in subsequent experiments to improve the problem.

TABLE II. DISTRIBUTION OF TEST RESULTS OF DIFFERENT PROJECTS.

Project	Actual code fragments /piece		Detect code fragments /piece		
	Source code	Variation insertion	Successful detection	Error detection	missed detection
pandas	16	20	34	0	2
scipy	46	51	92	3	5
django	42	45	74	1	13
pytorch	89	132	178	11	43
scikit-learn	14	15	28	0	1
keras	12	12	24	0	0

According to the recorded information of the variation insertion process, analyze the test results, found that the 414 detected clone segment of source code contains 412 real clone fragment. That is, these clone fragment come from 494 clone code fragments including source code fragment and variation insertion code fragment, and one detected clone fragment comes from noise code fragment. Calculate the recall and precision of the clone detection results

C. Experimental results and analysis

1) Clone code detection results and analysis

The final result of clone code detection is fed back in XML (extensible markup language) to provide data exchange basis for further analysis of clone code data. Figure 3 shows a partial results of the XML.

obtained by ICCV using formula (8) and formula (9). The recall of this experiment is $430 / 494 = 87.04\%$, precision is $412 / 414 = 96.38\%$. The precision of this experiment is relatively high due to the small proportion of noise fragments set in the data set in the total code clone fragment, and the low similarity between noise fragment and variation inserted code fragments. In the following work, the number of clone code fragment and noise code fragment in the data set will be continuously expanded.

In order to analyze the effect of ICCV on clone code detection of type-1, type-2 and type-3, this paper calculates the number of three types of clone code in the final test result according to the variation insertion record information in the data set, and respectively calculates the recall and precision of each type of clone detected by Jaccard and pHash, details are shown in table III.

TABLE III. RECALL AND PRECISION FOR DIFFERENT CLONE TYPES.

Type	Jaccard		pHash	
	recall	precision	recall	precision
Type-1	1.00	1.00	1.00	1.00
Type-2	0.65	0.96	0.88	1.00
Type-3	0.10	1.00	0.60	0.92

According to the results of Table 3, the Jaccard method can accurately detect all type-1 clone and 60% type-2 clone, while only 10% of type-3 can be detected. Using the pHash algorithm, 100% of type-1 clone, 88% of type-2 clone code, and 60% of type-3 clone can be detected. It can be concluded that using pHash algorithm to detect clone code has a more accurate and comprehensive effect than using Jaccard to detect, that is, using pHash algorithm to further detect clone that cannot be detected by Jaccard.

2) Visualization results and analysis

The visual results of detected clone code information are fed back in PDF form, which facilitates developers to analyze the clone data. Figure 4 shows a visualization of the clone code detected in django software.

Detected software Django, and extracted the detected clone information. There are 1639 nodes and 1150 edges, that is, 1639 clone fragments and 1150 clone relationships. Through the lines in the figure, the structural relationship between each clone code can be clearly seen; the larger the node, the higher the frequency of clone of the code fragment. Zoom in on the image to see the details between the nodes and the properties of the node. When maintaining

clone code, developers can prioritize these clone-prone code fragments for better management.

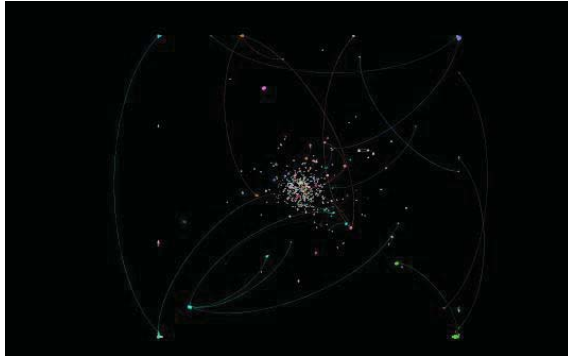


Figure 4. Clone code visual result

D. Comparative experiment and analysis

In order to verify the effectiveness of ICCV, this experiment was compared with NiCad under the same detection software and the same experimental environment. NiCad was selected because it has excellent performance in the evaluation and comparative study of current clone code detection methods [7][38]. Currently, there are few ways for clone code detection of Python language, while NiCad can detect clone in Python language and is widely recognized in the field of clone code. NiCad can detect clone of type-1, type-2, and part type-3.

Using the experimental evaluation data set and the mutation insertion related information constructed above, ICCV and NiCad were respectively used for clone detection, and different recall and precision were obtained as shown in table IV.

TABLE IV. RECALL AND ACCURACY FOR DIFFERENT METHOD.

Project	NiCad		ICCV	
	recall	precision	recall	precision
pandas	0.97	1.00	0.94	1.00
scipy	0.88	1.00	0.95	1.00
django	0.86	1.00	0.85	1.00
pytorch	0.76	0.98	0.81	0.98
scikit-learn	0.97	1.00	0.97	1.00
keras	1.00	1.00	1.00	1.00
Average	0.82	0.96	0.87	0.96

By comparing the experimental results, it is found that the two kinds of clone code detection methods have good detection effect on six softwares. Among the six tested software, both NiCad and ICCV were able to detect clone code with high precision, almost 100%. According to the respective average values of recall and accuracy, recall for clone code detected using ICCV were about 5 percentage points higher than those detected using NiCad. According to the clone information detected by human inspection, the precision of ICCV is comparable to the precision of NiCad. Although ICCV misses some real code clone fragments, ICCV can detect those that cannot be detected by NiCad. The results of the inspection and analysis personnel include the author herself, 5 school clone research experts and 6 enterprise software developers.

E. Experimental validity

The code preprocessing in this paper is mainly implemented through Python, code converted to image

using text processing tool Highlight 3.4.8 and call Python imgkit 1.0.1 package implementation, image processing and clone code detection based on image similarity and visualization use Python programming and image processing tool Python PIL, scientific calculation and analysis tool numpy 1.15.2 and network analysis software Gephi, and the rest of the work is realized with Python.

V. CONCLUSION

This paper proposes an image-based clone code detection and visualization method, detecting clone code based on semantic similarity of images, it is a new source code representation method, which provides a new perspective for clone code detection. Visualize the detected clone code, can highlight the key information in the data, improve the value of the data, facilitate the relevant personnel to analyze the clone data. Image representation source code fragments are used for clone detection, which lays a foundation for further research on deep learning.

The research work in this paper still has some shortcomings, for example, some clone code may be missed due to the limited number of lines of code selected, the detection algorithm can be further optimized, can only be tested for Python language projects and so on. In the future work, this paper will continue to study and solve these problems, and intend to conduct more in-depth research on clone code detection by combining deep learning techniques to achieve better results.

ACKNOWLEDGMENTS

WANG Yafang, born in 1994, M. S. candidate. Her research interests include software analysis, code analysis, etc. LIU Dongsheng, born in 1956, Ph. D., professor. His research interests include software analysis, code analysis, computer aided instruction, etc.

REFERENCES

- [1] Chen W K, Li B G and Gupta R. Code compact on of matching single-entry multiple-exit regions. [C]// International Static Analysis Symposium. Springer, 2003: 401-417.
- [2] Kim M, Sazawal V, Notkin D, et al. An empirical study of code clone genealogies[C]// 13th ACM SIGSOFT international symposium on Foundations of software engineering. ACM, 2005: 187-196.
- [3] Patenaude J F, Merlo E, Dagenais M, et al. Extending software quality assessment techniques to java systems[C]// Proceedings of the 7th International Workshop on Program Comprehension. IEEE, 1999:49-56.
- [4] Kamiya T, Kusumoto S, Inoue K. CCFinder: A Multilingual Token-Based Code Clone Detection System for Large Scale Source Code[J]. IEEE Transactions on Software Engineering, 2002, 28(7):654-670.
- [5] Rieger M, Stéphane Ducasse, Lanza M. Insights into System-Wide Code Duplication[C]// Conference on Reverse Engineering. IEEE, 2005: 100-109.
- [6] Roy C K, Cordy J R. A survey on software clone detection research. Queen's School of Computing TR, 2007, 541(115):64-68.
- [7] Roy C K, Cordy J R, Koschke R. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach [J]. Science of Computer Programming, 2009, 74(7):470-495.
- [8] Bellon S, Koschke R, Antoniol G, et al. Comparison and evaluation of clone detection tools [J]. IEEE Transactions on Software Engineering, 2007, 33(9):577-591.
- [9] Roy C K, Cordy J R. NICAD: Accurate Detection of Near-Miss Intentional Clones Using Flexible Pretty-Printing and Code Normalization[C]// Program Comprehension, 2008. ICPC 2008. The 16th IEEE International Conference on. IEEE, 2008:172-181.

- [10] Lee S. SDD: High Performance Code Clone Detection System for Large Scale Source Code[C]// Companion to the Acm Sigplan Conference on Object-oriented Programming. ACM, 2005:140-141.
- [11] Baker B S. On finding duplication and near-duplication in large software systems[C]// Proceedings of the 2nd IEEE Working Conference on Reverse Engineering, 1995. IEEE, 1995:86-95.
- [12] Ducasse, Stéphane, Rieger M, Demeyer S. A Language Independent Approach for Detecting Duplicated Code[C]// IEEE International Conference on Software Maintenance. IEEE, 1999:109-118.
- [13] Li Z, Lu S, Myagmar S, et al. CP-Miner: finding copy-paste and related bugs in large-scale software code[J]. IEEE Transactions on Software Engineering, 2006, 32(3):176-192.
- [14] Li L, Feng H, Zhuang W, et al. CCLearner: A Deep Learning-Based Clone Detection Approach[C]// 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE Computer Society, 2017:249-260.
- [15] Yuan Y, Guo Y. Boreas: an accurate and scalable token-based approach to code clone detection[C]// IEEE/ACM International Conference on Automated Software Engineering. IEEE, 2012:286-289.
- [16] Jiang L, Misherghi G, Su Z, et al. DECKARD: Scalable and Accurate Tree-Based Detection of Code Clones[C]// International Conference on Software Engineering. IEEE Computer Society, 2007:96-105.
- [17] Baxter I D, Yahin A, Moura L, et al. Clone detection using abstract syntax trees[C]// Conference on Reverse Engineering. IEEE, 2006:253-262.
- [18] Wei H H, Li M. Supervised Deep Features for Software Functional Clone Detection by Exploiting Lexical and Syntactical Information in Source Code[C]// International Joint Conferences on Artificial Intelligence Organization. IJCAI, 2017:3034-3040.
- [19] Krinke J. Identifying Similar Code with Program Dependence Graphs[C]//Conference on Reverse Engineering. IEEE, 2001:301-309.
- [20] Hummel B, Juergens E, Heinemann L, et al. Index-based code clone detection: incremental, distributed, scalable[C]//2010 IEEE International Conference on Software Maintenance. IEEE Computer Society, 2010:1-9.
- [21] Komondoor R, Horwitz S. Using slicing to identify duplication in source code[C]// 8th International Symposium on Static Analysis. Springer-Verlag, 2001:40-56.
- [22] Zhang Y L, Basit H A, Jarzabek S, et al. Query-based filtering and graphical view generation for clone analysis[C]//2008 IEEE International Conference on Software Maintenance. IEEE Computer Society, 2008:376-385.
- [23] ZHANG J J, WANG C H, ZHANG L P, et al. Clone code detection based on Levenshtein distance of token[J]. Journal of Computer Applications, 2015, 35(12):3536-3543.
- [24] McCormick W P, Lyons N I, Hutcheson K. Distributional properties of jaccard's index of similarity[J]. Communications in Statistics, 1992, 21(1):18.
- [25] YU T T, XU P N, JIANG Y, et al. Text Similarity Method Based on the Improved Jaccard Coefficient[J]. Computer Systems & Applications, 2017, 26(12):137-142.
- [26] TIAN X, ZHENG J, ZHANG Z P. Jaccard Text Similarity Algorithm Based on Word Embedding[J]. Computer Science, 2018, 45(07):186-189.
- [27] Ragkhitwetsagul C, Krinke J, Marnette B. A picture is worth a thousand words: Code clone detection based on image similarity[C]// IEEE International Workshop on Software Clones. IEEE Computer Society, 2018.
- [28] SONG B, JIANG W L, SUN T, et al. Image registration algorithm based on fast feature extraction and perceptual hash. Computer Engineering and Applications, 2018, 54(07):206-212.
- [29] LI D P, YANG C, JINAG Q, et al. A Client-Based Image Fuzzy Deduplication Method Supporting Proof of Ownership[J]. Chinese Journal of Computers, 2018, 41(06):1047-1063.
- [30] Jacomy M, Venturini T, Bastian M, et al. ForceAtlas2, a Continuous Graph Layout Algorithm for Handy Network Visualization Designed for the Gephi Software[J]. PLOS ONE, 2014, 9(6): e98679.
- [31] Hu Y F. Efficient and high quality force-directed graph drawing[J]. Mathematica Journal, 2006, 10: 37-71.
- [32] Noack A. Energy Models for Graph Clustering[J]. Journal of Graph Algorithms and Applications, 2007, 11(2):453-480.
- [33] Frick A, Ludwig A, Mehldau H. A Fast Adaptive Layout Algorithm for Undirected Graphs[C]// Proc DIMACS Int Work Graph Drawing. Springer-Verlag, 1994:388-403.
- [34] Svajlenko J, Roy C K. BigCloneEval: A Clone Detection Tool Evaluation Framework with BigCloneBench[C]// 2016 IEEE International Conference on Software Maintenance and Evolution (ICSME). IEEE, 2016:131-140.
- [35] Roy C K, Cordy J R. A Mutation/Injection-Based Automatic Framework for Evaluating Code Clone Detection Tools[C]// C3S2E Conference. ACM, 2008:157-166.
- [36] Roy C K, Cordy J R. A Mutation/Injection-Based Automatic Framework for Evaluating Code Clone Detection Tools[C]// C3S2E Conference. ACM, 2008:157-166.
- [37] SU X H, ZHANG F L. A Survey for Management-Oriented Code Clone Research[J]. Chinese Journal of Computers, 2018, 41(03):628-651.
- [38] Svajlenko J, Roy C K. Evaluating Modern Clone Detection Tools[C]// IEEE International Conference on Software Maintenance & Evolution. IEEE, 2014:321-330.